

CoDA-Synth

*Stylometric Detection of LLM-Rewritten Dark-Web Documents
with Traditional Machine Learning — Full Project + Google Colab Guide*

Finalised scope: public dark-web text corpora only (CoDA / DUTA). Two tasks — (A) human vs LLM-rewritten text and (B) original topic classification — using sparse features (word TF–IDF, character n-grams, stylometry) and classical classifiers (LinearSVC, logistic regression, Random Forest / XGBoost). Implemented end-to-end in Google Colab. No live Tor crawling.

Ethics lock: research and academic use of already-published, often-masked corpora. Do not collect new hidden-service pages. Do not reproduce operational criminal instructions. Synthetic class is created by paraphrasing public documents, not by training on live underground models.

1. What you are building

Underground markets increasingly use language models to polish listings and forum posts. Most published work classifies *topic* (drugs, financial, hacking, ...). This project asks a second question that you can answer with public data: can sparse stylometric features tell a human-authored dark-web document from an LLM paraphrase of the same document, without fine-tuning a transformer?

Research questions

- RQ1.** Can TF–IDF + character n-grams + cheap stylometry separate human CoDA/DUTA text from LLM rewrites of the same documents?
- RQ2.** Does adding stylometry improve source detection over TF–IDF alone?
- RQ3.** Does source-aware training hurt or help the original 10-way topic task?
- RQ4.** Does a detector trained on Model-A rewrites transfer to Model-B rewrites?

Suggested paper / report title

Stylometric Detection of LLM-Rewritten Dark-Web Documents with Sparse Features
Short name for files and slides: **CoDA-Synth**.

2. Deliverables

- A cleaned table: id, text, topic, source (human | llm), split, generator.
- Two trained classical models (source + topic) plus TF–IDF-only ablation.
- Metrics: source F1 / AUC; topic weighted F1; confusion matrices; feature-importance or top coefficients.
- A short report (this document is the method chapter).
- Optional: cross-generator stress test.

3. Data (public only)

Primary corpus: **CoDA** (Comprehensive Darkweb Annotations) — 10,000 documents in 10 topic categories, introduced by Jin et al., NAACL 2022. Hosted on Hugging Face as **s2w-ai/CoDA** under a research-use request (institutional email, purpose statement, ACM ethics). Documents are processed with mask identifiers for sensitive strings.

Fallback: **DUTA / DUTA-10K** (Al-Nabki et al.) — labeled Tor hidden-service pages, coarser and older categories. Use it if CoDA access is delayed. Do not mix label schemes in one classifier without a mapping table.

Source	Size (approx.)	Labels	Access
CoDA (Jin et al. 2022)	10,000 docs	10 topics	HF request: s2w-ai/CoDA

Source	Size (approx.)	Labels	Access
DUTA-10K	~10k domains	multi-class activities	authors / prior papers
Closed-market CSVs (GitHub)	varies	title, category	optional extra text only

Table 1. Allowed data sources. Prefer CoDA.

CoDA topic set (typical): Arms, Crypto, Drugs, Electronics, Financial, Gambling, Hacking, Pornography, Violence, Others. Confirm exact strings after you load the files — do not invent extra classes.

4. Ethics and legal frame

- Use corpora released for research. Cite Jin et al. (2022) / Al-Nabki et al. when you use their text.
- Do not attempt to unmask vendors, wallets, or victims. Keep CoDA masks intact.
- Do not scrape live .onion markets or forums for this project.
- Synthesis prompt must paraphrase existing public text only — no new criminal how-to, no new product invention.
- Store data in your private Google Drive; do not republish the raw CoDA dump.
- If a document looks like CSAM or trafficking, drop the row and do not inspect further. CoDA authors already filtered media.

5. Method overview

Pipeline: load public docs → light clean → paraphrase a disjoint 50% with a local/Colab open model (or a cheap API if your institution allows) → split by original document id → extract sparse features on train only → train two classifiers → ablate → write results.

Stage	Output	Tool in Colab
Load + clean	raw.csv	pandas, regex
Synthesize rewrites	coda_synth.csv	transformers pipeline or API
Id-safe split	train / val / test	sklearn train_test_split on ids
Features	sparse matrix + stylo	TfidfVectorizer, hstack
Classify	two models	LinearSVC, LogisticRegression, XGB
Evaluate	reports + plots	sklearn.metrics, matplotlib

Table 2. Pipeline stages.

6. Cleaning rules

- Strip residual HTML tags and collapse whitespace.
- Do not lowercase if you want case-ratio as a feature; keep a raw copy and a analysis copy.
- Keep slang, typos, and vendor voice — that is signal for both tasks.
- Drop documents with fewer than ~40 whitespace tokens.
- Leave CoDA mask tokens (e.g. entity placeholders) as-is.
- Columns to keep: doc_id, text, topic. Add source later.

7. Building the synthetic class

You do not have labeled jailbroken-market ads. That is fine. You create a controlled contrast: same facts, different writer. Randomly mark half of document ids for rewriting. The rewrite inherits topic. Human rows keep source=human; rewritten rows get source=llm and generator=.

Rewrite prompt (use this wording)

Rewrite the following document in a different wording and sentence rhythm. Keep the same facts, names that are already masked, and the same topic. Do not add products, prices, instructions, or new claims. Do not mention that you are an AI.

DOCUMENT:
{text}

If source-F1 later hits 0.99, the rewrites are too tidy. Mix two generators and two temperatures. That also gives you RQ4 for free.

Colab constraint: a 7B model on a free T4 is slow but workable for a few thousand short docs. If GPU quota is gone, use a small instruct model, or paraphrase only 1,500 docs for a pilot and state the sample size honestly.

8. Splits (do not leak pairs)

A document and its rewrite share content. If one is in train and the other in test, the classifier cheats on topic words. Split on **original doc_id** first (70 / 15 / 15), stratify by topic, then attach each id's human and llm rows to the same split.

9. Features (traditional ML)

9.1 Word TF-IDF

sklearn TfidfVectorizer: lowercase=True, ngram_range=(1, 2), min_df=3, max_df=0.9, max_features=50_000, sublinear_tf=True. Fit on train texts only.

9.2 Character TF-IDF

Second vectorizer: analyzer='char_wb', ngram_range=(3, 5), min_df=3, max_features=30_000. Character n-grams pick up LLM smoothness, punctuation habits, and tokenisation artefacts.

9.3 Stylometric vector (dense, then scaled)

Feature	Why it may help
Token count	Rewrites often lengthen or flatten text
Mean / std word length	LLM diction vs slang stubs
Mean sentence length	Human fragments vs polished clauses
Type-token ratio	Lexical diversity
Hapax ratio	One-off jargon
Stopword ratio	Function-word fingerprint
Punctuation rate	Over-neat commas vs raw listings
Digit ratio	Prices and versions
Uppercase ratio	Shouting titles
Exclaim / question rate	Marketing tone

Table 3. Cheap stylometry. Compute per document with regex + split; no extra APIs.

Stack: `scipy.sparse.hstack([X_word, X_char, StandardScaler().fit_transform(X_stylo)])`. Scaler fitted on train stylo only.

10. Classifiers

Train **two models**, same feature matrix. Do not build a neural dual head.

Model	Role	Notes
LinearSVC	Primary source + topic baseline	class_weight='balanced', dual=True or False by n_samples
LogisticRegression	Probabilities + AUC + coefficients	solver='saga', max_iter=2000
RandomForestClassifier	Non-linear check	n_estimators=300, n_jobs=-1
XGBClassifier	Strongest classical option	eval_metric='logloss' or 'mlogloss'

Table 4. Models. Start with LinearSVC; add XGBoost if time remains.

Hyperparameters: GridSearchCV on validation F1, small grid only (C in {0.5, 1, 2, 4} for linear models; max_depth in {3, 5, 8} for trees). Fix random_state=42. Report three seeds if you have time.

11. Metrics and ablations

- Source task: precision, recall, F1 (positive = llm), ROC-AUC from logistic / XGB probabilities. LinearSVC: CalibratedClassifierCV or decision_function AUC.
- Topic task: accuracy, weighted F1, macro F1, confusion matrix.
- Ablation A: word TF-IDF only.
- Ablation B: word + char.
- Ablation C: word + char + stylo (full).
- RQ3 check: topic F1 of a topic-only model vs the same model trained on the mixed human+llm set.
- RQ4: train source detector on generator A, test on generator B.

12. Google Colab — environment

Use a standard Colab runtime. GPU is optional and only needed for the rewrite step. CPU is enough for TF-IDF and LinearSVC on 10–20k short documents.

12.1 Folder layout on Drive

```
MyDrive/coda_synth/
  data/          raw.csv, coda_synth.csv, splits.parquet
  models/        source_svc.joblib, topic_svc.joblib, vectorizers.joblib
  outputs/       reports, png plots
  notebooks/     coda_synth_colab.ipynb
```

12.2 First cell — install and mount

```
!pip -q install scikit-learn==1.5.2 pandas numpy scipy xgboost joblib matplotlib seaborn
# optional for local rewrite on GPU:
# !pip -q install transformers accelerate bitsandbytes

from google.colab import drive
drive.mount('/content/drive')
ROOT = '/content/drive/MyDrive/coda_synth'
import os
for d in ['data', 'models', 'outputs']:
    os.makedirs(f'{ROOT}/{d}', exist_ok=True)
```

12.3 Load CoDA after HF access is granted

After Hugging Face accepts the CoDA request, create a token (read scope) and paste it into Colab secrets as HF_TOKEN. Do not hard-code tokens in the notebook you submit.

```

from huggingface_hub import login
from google.colab import userdata
login(token=userdata.get('HF_TOKEN'))

from datasets import load_dataset
ds = load_dataset('s2w-ai/CoDA') # inspect splits / columns immediately
print(ds)
print(ds['train'].column_names) # names vary; map them to doc_id, text, topic

```

If the dataset is a WebDataset of files rather than a simple table, iterate examples and write a CSV. The official page states research-only terms — keep the acknowledgement cell in the notebook.

12.4 Clean

```

import re, pandas as pd

TAG = re.compile(r'<[^>]+>')
WS = re.compile(r'\s+')

def clean_text(s):
    s = TAG.sub(' ', str(s))
    s = WS.sub(' ', s).strip()
    return s

df['text'] = df['text'].map(clean_text)
df['n_tok'] = df['text'].str.split().str.len()
df = df[df['n_tok'] >= 40].copy()
df.to_csv(f'{ROOT}/data/raw.csv', index=False)

```

13. Colab — synthesis cell

Pilot path (recommended first): paraphrase 1,000–2,000 docs. Full path: half of remaining ids. Example uses a small instruct model via transformers if a GPU is attached; otherwise call an institution-approved API with the same prompt.

```

import numpy as np
rng = np.random.default_rng(42)
ids = df['doc_id'].unique()
rewrite_ids = set(rng.choice(ids, size=int(0.5 * len(ids)), replace=False))

PROMPT = '''Rewrite the following document in a different wording and sentence rhythm.
Keep the same facts and the same topic. Do not add new claims.\n\nDOCUMENT:\n'''

# attach your generate(prompt) function here
rows = []
for _, r in df.iterrows():
    rows.append(**r.to_dict(), 'source': 'human', 'generator': 'original')
    if r['doc_id'] in rewrite_ids:
        new_text = generate(PROMPT + r['text'][:3000]) # truncate long pages
        rec = r.to_dict()
        rec.update({'text': new_text, 'source': 'llm', 'generator': 'MODEL_NAME'})
        rows.append(rec)
syn = pd.DataFrame(rows)
syn.to_csv(f'{ROOT}/data/coda_synth.csv', index=False)

```

Sanity check after synthesis: print two side-by-side examples. If the model refuses or empties the page, retry once and otherwise drop that pair.

14. Colab — id-safe split

```
from sklearn.model_selection import train_test_split

meta = syn.drop_duplicates('doc_id')[['doc_id', 'topic']]
id_train, id_tmp = train_test_split(
    meta['doc_id'], test_size=0.30, random_state=42,
    stratify=meta['topic'])
id_val, id_test = train_test_split(
    id_tmp, test_size=0.50, random_state=42,
    stratify=meta.set_index('doc_id').loc[id_tmp, 'topic'])

def tag(frame, ids, name):
    out = frame[frame.doc_id.isin(ids)].copy()
    out['split'] = name
    return out

full = pd.concat([
    tag(syn, id_train, 'train'),
    tag(syn, id_val, 'val'),
    tag(syn, id_test, 'test'),
], ignore_index=True)
full.to_parquet(f'{ROOT}/data/splits.parquet')
```

15. Colab — features and training

```

import re, numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer, ENGLISH_STOP_WORDS
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, roc_auc_score, f1_score
from scipy.sparse import hstack, csr_matrix
import joblib

STOP = set(ENGLISH_STOP_WORDS)

def stylo(text):
    t = text or ''
    toks = re.findall(r"\b\w+\b", t.lower())
    n = max(len(toks), 1)
    sents = re.split(r'[!?.]+', t)
    sents = [s for s in sents if s.strip()]
    ns = max(len(sents), 1)
    uniq = len(set(toks))
    hap = sum(1 for w in set(toks) if toks.count(w) == 1)
    return np.array([
        n,
        np.mean([len(w) for w in toks]) if toks else 0,
        np.mean([len(s.split()) for s in sents]) if sents else 0,
        uniq / n,
        hap / n,
        sum(w in STOP for w in toks) / n,
        sum(ch in '.,;:!?-()' for ch in t) / max(len(t), 1),
        sum(ch.isdigit() for ch in t) / max(len(t), 1),
        sum(ch.isupper() for ch in t) / max(len(t), 1),
        t.count('!') / ns,
        t.count('?') / ns,
    ], dtype=float)

tr = full[full.split=='train']; va = full[full.split=='val']; te = full[full.split=='test']

word = TfidfVectorizer(ngram_range=(1,2), min_df=3, max_df=0.9,
                      max_features=50000, sublinear_tf=True)
char = TfidfVectorizer(analyzer='char_wb', ngram_range=(3,5),
                      min_df=3, max_features=30000)
Xw_tr, Xw_va, Xw_te = word.fit_transform(tr.text), word.transform(va.text), word.transform(te.text)
Xc_tr, Xc_va, Xc_te = char.fit_transform(tr.text), char.transform(va.text), char.transform(te.text)

S_tr = np.vstack(tr.text.map(stylo))
S_va = np.vstack(va.text.map(stylo))
S_te = np.vstack(te.text.map(stylo))
scaler = StandardScaler()
S_tr, S_va, S_te = scaler.fit_transform(S_tr), scaler.transform(S_va), scaler.transform(S_te)

def pack(w, c, s):
    return hstack([w, c, csr_matrix(s)]).tocsr()
X_tr, X_va, X_te = pack(Xw_tr, Xc_tr, S_tr), pack(Xw_va, Xc_va, S_va), pack(Xw_te, Xc_te, S_te)

y_src_tr = (tr.source=='llm').astype(int)
y_src_va = (va.source=='llm').astype(int)
y_src_te = (te.source=='llm').astype(int)
le = LabelEncoder()
y_top_tr = le.fit_transform(tr.topic)
y_top_va = le.transform(va.topic)
y_top_te = le.transform(te.topic)

src = LinearSVC(class_weight='balanced', random_state=42)
src.fit(X_tr, y_src_tr)
print('SOURCE val\n', classification_report(y_src_va, src.predict(X_va), digits=3))

top = LinearSVC(class_weight='balanced', random_state=42)
top.fit(X_tr, y_top_tr)
print('TOPIC val\n', classification_report(y_top_va, top.predict(X_va), digits=3, target_names=le.classes_))

joblib.dump({'word':word, 'char':char, 'scaler':scaler, 'le':le, 'src':src, 'top':top},
            f'{ROOT}/models/bundle.joblib')

```

16. Colab — evaluation and plots

```
from sklearn.metrics import ConfusionMatrixDisplay, RocCurveDisplay
import matplotlib.pyplot as plt

pred_s = src.predict(X_te)
pred_t = top.predict(X_te)
print(classification_report(y_src_te, pred_s, target_names=['human','llm'], digits=3))
print('topic weighted F1', f1_score(y_top_te, pred_t, average='weighted'))

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
ConfusionMatrixDisplay.from_predictions(y_src_te, pred_s, display_labels=['human','llm'], ax=ax[0], colorbar=False)
ConfusionMatrixDisplay.from_predictions(y_top_te, pred_t, display_labels=le.classes_, ax=ax[1], colorbar=False, xticks_rotation=45)
ax[0].set_title('Source'); ax[1].set_title('Topic')
fig.tight_layout()
fig.savefig(f'{ROOT}/outputs/confusion.png', dpi=160)
```

Ablation: rebuild X with word only, then word+char, repeat LinearSVC, store three F1 numbers in a small table. That table is the centre of your results section.

17. Week plan (Colab reality)

Day	Work
Mon	HF access, Drive folders, load + clean → raw.csv
Tue	Pilot rewrite of 500–1000 docs; spot-check pairs
Wed	Full (or accepted) synth + id-safe split
Thu	Features + LinearSVC + logistic; val metrics
Fri	Ablations, XGBoost if needed, plots
Sat	Write results, limits, citations

Table 5. Six-day schedule.

18. What to write in the report

- Problem: topic classifiers ignore authorship shift toward generated underground copy.
- Constraint: no live collection; public CoDA/DUTA + controlled paraphrase.
- Method: sparse word/char TF–IDF + 11 stylo features; LinearSVC / logistic / trees.
- Leakage control: split by original id.
- Results: source F1, topic weighted F1, ablation deltas, optional cross-generator drop.
- Limits: paraphrase ≠ WormGPT-style tools; English-heavy CoDA; masks change surface form.
- Ethics: research terms, no redistribution of raw dump, no operational targeting.

19. Limitations you should say out loud

A detector that sees your own paraphrases may not see a specialised uncensored model sold on forums. State that CoDA-Synth is a **proxy benchmark**. Character n-grams can overfit generator punctuation. Topic labels on rewritten text assume the model obeyed “keep the topic” — spot-check a sample. Free Colab sessions disconnect; always write parquet to Drive after each stage.

20. Citations to include

Jin, Y., Jang, E., Lee, Y., Shin, S., & Chung, J.-W. (2022). Shedding New Light on the Language of the Dark Web. NAACL.

Al-Nabki, M. W., Fidalgo, E., Alegre, E., & Fernández-Robles, L. (2017/2019). DUTA / DUTA-10K darknet usage text addresses.

Jin et al. (2023). DarkBERT: A Language Model for the Dark Side of the Internet. ACL. (related work — you are *not* required to run DarkBERT.)

Plus any LLM you used for paraphrases (model card + version date).

21. Colab checklist before you submit

- Notebook runs top-to-bottom on a fresh runtime after Drive mount.
- Secrets used for HF token; no keys in git.
- splits.parquet exists; train/val/test counts printed.
- Ablation table saved as CSV in outputs/.
- Confusion matrices readable (rotated topic labels).
- README cell lists CoDA licence terms.
- You can explain every feature in Table 3 without notes.

22. One-paragraph pitch (for a supervisor)

This project studies whether classical sparse features can detect LLM paraphrases of public dark-web documents while still classifying the original CoDA topic. Using only research corpora and a controlled rewrite protocol, we train LinearSVC and logistic models on word TF-IDF, character n-grams, and stylometry, with document-id splits to block pair leakage. The work is fully implementable in Google Colab on CPU after an optional GPU paraphrase pass, and it treats generated underground text as a measurement problem rather than an operational collection problem.

End of guide. Scope is frozen: public data, traditional ML, two tasks, Colab. If CoDA access lags, start the pipeline on a 2,000-row pilot from DUTA and swap files later — the code does not change.